

DSA LAB-6

2.3.26

NAME: Sanjana Pal

USN: 1RUA25BCA0096

1. Inserting a node at the beginning of a doubly-linklist:

Code-

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Node {
4     int data;
5     struct Node *prev, *next;
6 };
7 void insertAtBeginning(struct Node** head, int val) {
8     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
9     newNode->data = val;
10    newNode->next = *head;
11    newNode->prev = NULL;
12    if (*head != NULL) (*head)->prev = newNode;
13    *head = newNode;
14 }
15 int main() {
16     struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
17     struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
18     struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
19     struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
20    n1->data = 10; n1->prev = NULL; n1->next = n2;
21    n2->data = 20; n2->prev = n1; n2->next = n3;
22    n3->data = 30; n3->prev = n2; n3->next = n4;
23    n4->data = 40; n4->prev = n3; n4->next = NULL;
24    struct Node* head = n1;
25    insertAtBeginning(&head, 5);
26    struct Node* temp = head;
27    while(temp != NULL) {
28        printf("%d <-> ", temp->data);
29        temp = temp->next;
30    }
31    printf("NULL\n");
32    return 0;
33 }
```

Output-

Output

5 <-> 10 <-> 20 <-> 30 <-> 40 <-> NULL

2. Deleting a node at the end of a doubly linked list:

Code-

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Node {
4     int data;
5     struct Node *prev, *next;
6 };
7 void insertAtEnd(struct Node* head, int val) {
8     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
9     newNode->data = val;
10    newNode->next = NULL;
11    struct Node* temp = head;
12    while(temp->next != NULL) temp = temp->next;
13    temp->next = newNode;
14    newNode->prev = temp;
15 }
16 int main() {
17     struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
18     struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
19     struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
20     struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
21     n1->data = 10; n1->prev = NULL; n1->next = n2;
22     n2->data = 20; n2->prev = n1; n2->next = n3;
23     n3->data = 30; n3->prev = n2; n3->next = n4;
24     n4->data = 40; n4->prev = n3; n4->next = NULL;
25     struct Node* head = n1;
26     insertAtEnd(head, 50);
27     struct Node* temp = head;
28     while(temp != NULL) {
29         printf("%d <-> ", temp->data);
30         temp = temp->next;
31     }
32     printf("NULL\n");
33     return 0;
34 }
```

Output-

```
Output
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL

=== Code Execution Successful ===
```

3.Inserting a node in between a doubly link-list:(pos=3)

Code-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *prev, *next;
};
void insertAtPos3(struct Node* head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    struct Node* temp = head;
    temp = temp->next;
    newNode->next = temp->next;
    newNode->prev = temp;
    if(temp->next != NULL) temp->next->prev = newNode;
    temp->next = newNode;
}
int main() {
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 10; n1->prev = NULL; n1->next = n2;
    n2->data = 20; n2->prev = n1; n2->next = n3;
    n3->data = 30; n3->prev = n2; n3->next = n4;
    n4->data = 40; n4->prev = n3; n4->next = NULL;
    struct Node* head = n1;
    insertAtPos3(head, 25);
    struct Node* temp = head;
    while(temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
    return 0;
}
```

Output-

Output

10 <-> 20 <-> 25 <-> 30 <-> 40 <-> NULL

=== Code Execution Successful ===

Deletion at start

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node* prev;
7      struct Node* next;
8  };
9
10 void deleteAtStart(struct Node** head) {
11     if (*head == NULL) return;
12     struct Node* temp = *head;
13     *head = (*head)->next;
14     if (*head != NULL) {
15         (*head)->prev = NULL;
16     }
17     free(temp);
18 }
19
20 int main() {
21     struct Node *n1 = malloc(sizeof(struct Node)), *n2 = malloc
        (sizeof(struct Node));
22     n1->data = 20; n1->prev = NULL; n1->next = n2;
23     n2->data = 30; n2->prev = n1; n2->next = NULL;
24     struct Node* head = n1;
25 }
```

Output

30

Deletion at end

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node* prev;
7      struct Node* next;
8  };
9
10 void deleteAtEnd(struct Node** head) {
11     if (*head == NULL) return;
12     struct Node* temp = *head;
13     if (temp->next == NULL) {
14         *head = NULL;
15         free(temp);
16         return;
17     }
18     while (temp->next != NULL) {
19         temp = temp->next;
20     }
21     temp->prev->next = NULL;
22     free(temp);
23 }
24
25 int main() {
26     struct Node *n1 = malloc(sizeof(struct Node)), *n2 = malloc
```

```

24
25 int main() {
26     struct Node *n1 = malloc(sizeof(struct Node)), *n2 = malloc
        (sizeof(struct Node));
27     n1->data = 20; n1->prev = NULL; n1->next = n2;
28     n2->data = 30; n2->prev = n1; n2->next = NULL;
29     struct Node* head = n1;
30
31     deleteAtEnd(&head);
32
33     struct Node* temp = head;
34     while (temp != NULL) {
35         printf("%d ", temp->data);
36         temp = temp->next;
37     }
38     return 0;
39 }

```

Output

20

Deletion at position

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node* prev;
7      struct Node* next;
8  };
9
10 void deleteAtPosition(struct Node** head, int pos) {
11     if (*head == NULL || pos <= 0) return;
12     struct Node* temp = *head;
13     if (pos == 1) {
14         *head = temp->next;
15         if (*head != NULL) (*head)->prev = NULL;
16         free(temp);
17         return;
18     }
19     for (int i = 1; temp != NULL && i < pos; i++) {
20         temp = temp->next;
21     }
22     if (temp == NULL) return;
23     if (temp->next != NULL) {
24         temp->next->prev = temp->prev;
25     }
26     if (temp->prev != NULL) {
```

```

23     if (temp->next != NULL) {
24         temp->next->prev = temp->prev;
25     }
26     if (temp->prev != NULL) {
27         temp->prev->next = temp->next;
28     }
29     free(temp);
30 }
31
32 int main() {
33     struct Node *n1 = malloc(sizeof(struct Node)), *n2 = malloc
        (sizeof(struct Node)), *n3 = malloc(sizeof(struct Node));
34     n1->data = 20; n1->prev = NULL; n1->next = n2;
35     n2->data = 30; n2->prev = n1; n2->next = n3;
36     n3->data = 40; n3->prev = n2; n3->next = NULL;
37     struct Node* head = n1;
38
39     deleteAtPosition(&head, 2);
40
41     struct Node* temp = head;
42     while (temp != NULL) {
43         printf("%d ", temp->data);
44         temp = temp->next;
45     }
46     return 0;
47 }

```

Output

20 40

=== Code Execution Successful ===